

Claude Code for Economic Research

A Complete Setup Guide

Guido Ardizzone

PhD Student, IHEID, Geneva

*Claude Code doesn't replace your judgment —
it removes the friction between your judgment and its execution.*

Contents

1	What Is Claude Code?	2
2	Installation & Access	2
3	How Claude Code Works	3
3.1	Built-in Tools	4
4	Customizing Claude Code	4
4.1	The .claude/ global folder	4
4.2	How Claude loads context: the two-layer system	5
4.3	Global CLAUDE.md in detail	5
4.4	Project-specific CLAUDE.md	6
4.5	Settings files	6
4.6	Skills — Reusable Playbooks	7
4.7	Agents — Parallel Workers	8
4.8	Rules — Auto-loaded Protocols	10
4.9	The full directory tree	10
5	Workflows & Best Practices	11
5.1	Session Continuity — Log Files and Handoffs	11
5.2	Privacy & Data Protection	12
5.3	Best Practices	13
6	A Complete Example: Guido’s Setup	14
6.1	The multi-layer flow in action	14
6.2	Directory structure	14
6.3	Selected skills, agents, and rules	15
7	Pricing & Token Consumption	15
7.1	Choosing a model	15
7.2	Two ways to pay	16
7.3	What consumes the most tokens?	16
7.4	Tips to reduce costs	17
8	Resources & Community	17
8.1	Official documentation	17
8.2	Learning & training	17
8.3	Skills & workflows	17
8.4	People to follow on X	17

1. What Is Claude Code?

Claude Code is not a chatbot. It is an **agent that acts**. Built by Anthropic, it lives in your terminal (or browser) and reads, writes, searches, and executes code **autonomously** on your machine. You give it a task in plain English, and it carries it out: reading your files, running your scripts, and modifying your code as needed. Equivalents exist by OpenAI (Codex) and Open Source (Code Llama, Gemini Pro, etc.); this guide focuses entirely on Claude Code.

	Regular chatbot	Claude Code
File access	Sees only what you paste	Reads your entire project
Execution	Cannot run your code	Runs Stata, R, Python, L ^A T _E X
Context	Forgets project structure	Understands project context
Workflow	You do all the work	Works while you think

The key distinction between Claude Code and the Regular Chatbot is that you have **full file access** and **execution capabilities**. This means you can delegate entire workflows to Claude Code. By setting it up properly you avoid repeating everytime context and rules. You write them once, and they are automatically available in every future session. Also, you can customize it as much as you want to your personal needs and conventions. It is not a black box. You can teach it your coding style, your folder structure, and your research process. It becomes your personal research assistant that understands you and works the way you want.

2. Installation & Access

There are four ways to use Claude Code. All use the same underlying engine and read the same configuration files. You can switch between them at any time.

Option A: Terminal (CLI)

This is the most powerful and flexible interface. The full installation instructions are in the official quickstart guide: <https://code.claude.com/docs/en/quickstart>. Everything is done from the terminal, no need to open a browser or download anything manually. You install Node.js, then Claude Code, and you are ready to go.

Tip

Let the chatbot walk you through it. Open Claude in the browser (<https://claude.ai>) and say: *“I want to install Claude Code on my Mac — guide me step by step.”* It will give you the exact terminal commands for your system and help you troubleshoot any issues. This is often the fastest and least intimidating way to get set up.

Option B: Desktop App

Download the Claude desktop app from <https://claude.ai>. It is available for Mac and Windows. The app provides a visual interface with the same terminal capabilities underneath. You get a chat-like window, but Claude still has full access to your files and can run commands.

Option C: Browser (claude.ai Code mode)

Go to <https://claude.ai> and switch to Code mode. No installation needed. You get a visual interface where you can drag and drop files. Claude has the same tools and file access as the

terminal version. Best for: getting started, visual workflows.

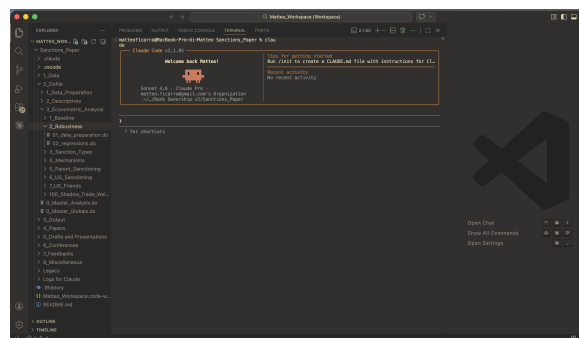
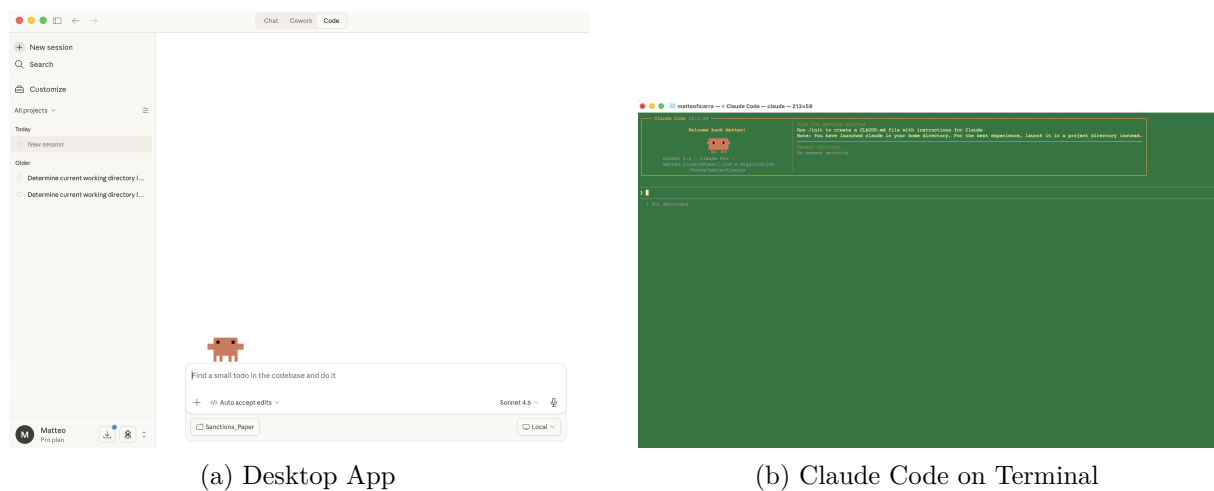
Option D: IDE Extensions

Claude Code is available as an extension for **VS Code** and **JetBrains** IDEs (IntelliJ, PyCharm, etc.). It runs inside your editor with direct access to your open files and project structure.

Which should I pick?

All four interfaces read the same configuration and act on the same files. The terminal CLI is the most powerful and scriptable. The desktop app and browser are more visual and beginner-friendly. Pick whichever fits your workflow; you can switch anytime. Once you become proficient, we recommend to use the Terminal within VS Code: you get the power of the CLI and you visualize the file structure and code as you work.

Figure 1: Claude Code Interface



(c) Claude Code on VS Code

3. How Claude Code Works

There is no single “right” way to set up and use Claude Code. What follows is how I use it, but from talking to colleagues I have learned that everyone develops different habits and workflows. Don’t feel constrained to do exactly the same—experiment and find what works best for you.

Now that we have installed Claude Code and have it running, let’s understand how it works under the hood. This will help you use it more effectively and customize it to your needs.

3.1. Built-in Tools

Claude Code has about ten built-in tools. You will use three of them daily; the rest are loaded on demand when needed.

Tip

Try to remember what the main tools do. Claude Code runs these tools on its own, but will ask you for permission. If you learn what they do, you avoid undesired outcomes!

The big three

Tool	What it does	Example
Read / Write / Edit	File I/O: read, create, and modify files	Read a <code>.do</code> file, edit a line, write a new <code>.tex</code> file
Bash	Run any shell command	Execute Stata in batch mode, compile L ^A T _E X, run Python
Grep / Glob	Search file contents and file names	Find all <code>.dta</code> files, search for a variable across do-files

Secondary tools (loaded on demand)

Tool	Purpose
Agent	Spawn a separate Claude instance to work in parallel
Skill	Invoke a reusable playbook (e.g. <code>/stata-regression</code>)
WebSearch	Search the web for documentation, papers, references
WebFetch	Download a specific URL
Task / Plan	Break work into steps, enter plan mode for complex tasks

Skills and agents control *how* these tools are used. They do not add new capabilities; they add structure and consistency to how Claude approaches your tasks.

4. Customizing Claude Code

When you install Claude Code, it creates a folder called `.claude/` on your machine. Think of it as Claude's home base — it is typically located in your system's home directory alongside other application folders. Every time you start a session, Claude reads this folder to understand who you are, what you are working on, and what rules to follow. From this `.claude/` folder, you teach Claude how to navigate your laptop: which folder is for which project, which rules to follow, and how to execute code. **Thirty minutes of initial setup saves hours of repeated instructions.**

4.1. The `.claude/` global folder

The global `.claude/` folder contains all the files that Claude reads at the start of *every* session, regardless of which project you work on. The most important file inside it is the `CLAUDE.md`, a Markdown file that serves as the starting point for Claude. It should be structured to provide comprehensive context: who you are, what tools you use, what your critical rules are, and what projects you are working on.

But the global folder contains more than just `CLAUDE.md`. It also holds your **skills**, **agents**, and **rules**, three types of customizations that we will cover in detail later in this section. For now, keep in mind that all of these live inside `~/.claude/` and are available in every session.

NB: `~/.claude/` is a “hidden” folder in your computer. On Mac, you can find it with `⌘ + ⌥ + I`. On Windows, click on “View” and show “Hidden items”.

4.2. How Claude loads context: the two-layer system

The key idea is simple: you have **global instructions** (loaded every session) and **project-specific instructions** (loaded when you pick a project). Here is how a typical session starts:

1. Claude reads your **global CLAUDE.md** (`~/.claude/CLAUDE.md`). It learns who you are, your tools, your critical rules, and your list of projects.
2. You tell Claude which folder you want to work in (e.g., OneDrive, Dropbox) and which specific project to focus on. You can have multiple projects in the same folder, but you need to specify which one you want to work on in that session. Your global `CLAUDE.md` can include a **session checklist** that automates this, for example telling Claude to ask: “Which project are we working in today?”
3. You answer (e.g., “Project 1”). Claude reads that project’s **local CLAUDE.md** and learns the project boundary, research focus, folder structure, and key files to read. This is important because it loads the right context *and* sets boundaries: Claude will not touch files outside that project folder (more on this in the privacy section).
4. Claude reads any **referenced context files** (e.g., a research plan, a paper outline, or a handoff file from the last session) and is ready to work.

Rules (from `~/.claude/rules/`) are loaded automatically in the background at every session. You never need to invoke them. **Skills** and **agents** are loaded on demand when you need them.

The result: Claude has full context in under 30 seconds. You write each piece of information once, and it is automatically available in every future session. It may sound complex at first, but the mental model is straightforward. Global instructions are always loaded; project-specific instructions are loaded when you pick a project. All the layers speak to each other automatically.

4.3. Global CLAUDE.md in detail

This is your master configuration file, loaded at the start of every session regardless of which project you choose. A good global `CLAUDE.md` has three layers:

Layer	Content	Example
Identity	Your role, tools, research context	“PhD candidate in economics. Tools: Stata, L ^A T _E X, R.”
Guardrails	Critical rules, coding conventions	“NEVER delete data files. Raw data is read-only.”
Context	Active projects, folders, phases	Table of projects with folder paths and status

Here is what a typical global `CLAUDE.md` looks like:

```
# CLAUDE.md -- Global Operating Rules
## Who I Am
PhD candidate in economics. Tools: Stata, LaTeX, R.
Working station: my_laptop.
```

```
## Critical Rules
0. Always work in OneDrive folder
1. NEVER delete data files (.dta, .xlsx, .csv)
2. NEVER delete program files (.do, .R, .py)
3. Raw data is sacred -- never modify files in Raw_Data/

## Active Projects
| Project | Folder | Phase |
| My JMP | 1-JM_Paper/ | Empirical analysis |
| Side project | 2-Side_Project/ | Data preparation |

## Session Checklist
At the start of each session:
1. Greet the user
2. Ask which project we are working in today
3. Read that project's .claude/CLAUDE.md
4. Read the project's handoff file
```

Tip

You do not have to write this from scratch. Start a session with Claude and say: *“Help me write a CLAUDE.md for my setup.”* Claude will interview you about your role, tools, projects, and rules, and then draft the file for you.

4.4. Project-specific CLAUDE.md

Each project gets its own CLAUDE.md inside a .claude/ subfolder within the project directory. This file complements the global one with project-specific information:

- **Boundary:** “Stay within ./Project_1/. Never navigate above this directory.” This is crucial: it prevents Claude from accidentally modifying files in other projects.
- **Research focus:** What the project is about, the research question, key data sources.
- **Important directories:** A table showing which folders exist, what they contain, and whether Claude is allowed to modify them.
- **Key context files:** Files that Claude should read at the start of each session (e.g., a research plan, a paper outline, the latest handoff).

Example:

```
# Project: My JMP

## Boundary
Stay within ./1-JM_Paper/. Never navigate above this directory.

## Research Focus
Job market paper on sovereign-bank transmission in Portugal.

## Important Directories
| Directory | Purpose | Modify? |
| Data/Raw_Data/ | Original source data | NEVER |
| Data/Final_Data/ | Analysis-ready | With caution |
| Codes/ | Analysis scripts | Ask before deleting |
| Drafts/ | Working papers | Active |
```

4.5. Settings files

~/.claude/settings.json controls global permissions: what Claude is allowed to do by default. You can allow specific tools, deny access to certain paths, and configure hooks:

```
{
  "permissions": {
    "allow": ["Read", "Edit", "Write", "Glob", "Grep",
      "Bash(stata-mp:*)", "Bash(pdflatex:*)"],
    "deny": ["Read(/Raw_Data/**)"]
  }
}
```

`project/.claude/settings.local.json` provides project-level overrides. For example, you might allow Stata execution only in specific projects. Again, use Claude to update this features.

4.6. Skills — Reusable Playbooks

A skill is essentially a saved prompt for a specific task. Think of it as writing down, once, exactly how you want Claude to handle a particular workflow: which conventions to follow, what output format to use, where to save files. Once it is written, you never need to re-explain those instructions. Each skill lives in its own folder inside `~/.claude/skills/` and contains:

- **SKILL.md** (required): A Markdown file with a short header block (name and description between `---` markers) followed by the full instructions. Typically the only thing needed
- **scripts/** (optional): Executable code (Python, Bash, etc.) for deterministic tasks.
- **references/** (optional): Documentation that Claude can load as needed.
- **assets/** (optional): Templates, fonts, icons used in output.

How skills work: progressive disclosure

Skills use a three-level system to minimize token usage:

1. **Frontmatter** (name + description): Always loaded into Claude's context (~100 words). This is what Claude uses to decide whether the skill is relevant to your current request.
2. **SKILL.md body**: Loaded only when the skill is actually triggered. Contains the full instructions.
3. **Bundled files** (scripts, references): Loaded on demand only when the skill needs them during execution.

How to invoke a skill

Type `/skill-name` in your conversation. For example:

```
/stata-regression
```

Claude loads the skill's full instructions and follows them, using your naming conventions, your fixed effects specifications, your output format, and your save locations. Every time.

How to create a skill manually

Create a folder in `~/.claude/skills/` with a **SKILL.md** file. The file must start with a header block (between `---` markers) containing at minimum a **name** and a **description**:

```
---
name:  stata-regression
description:  Run panel regressions with reghdfe, export tables to .tex
---

# Stata Regression Skill
## When to use
When the user asks to run regressions or create regression tables.
## Conventions
```



```

- Use reghdfe for all panel regressions
- Cluster standard errors at the firm level by default
- Export tables with esttab in booktabs format
- Save .tex files to the project's output/tables/ folder

## Steps
1. Load the data per the user's instructions
2. Run the regression with the specified fixed effects
3. Format the table with esttab
4. Save to the output folder
5. Print diagnostics to the terminal

```

The Skill Creator (/skill-creator)

You do not need to write skills entirely by hand. Anthropic provides an official **skill-creator** tool that builds skills for you through a structured, interactive workflow:

1. **Interview:** The skill-creator asks you what the skill should do, when it should trigger, what the expected outputs are, and what edge cases to handle.
2. **Write:** It drafts the `SKILL.md` with the header block and detailed instructions.
3. **Test:** It creates evaluation prompts and runs them in two modes, *with* the skill and *without* (baseline), to measure how much the skill improves Claude's output.
4. **Iterate:** Based on test results, it refines the instructions and re-tests.
5. **Optimize:** It fine-tunes the skill's description for better triggering accuracy, ensuring the skill activates when it should and stays quiet when it shouldn't.

To use it, type `/skill-creator` and follow the prompts. Expect about 15–30 minutes to build and test your first working skill.

Anthropic's official guide, "*The Complete Guide to Building Skills for Claude*", covers everything from fundamentals to testing and distribution. It is available at <https://github.com/anthropics/skills>.

Example skills

Skill	What it does
<code>/stata-regression</code>	Runs <code>reghdfe</code> regressions with your standard FE specs, exports <code>.tex</code> tables
<code>/keeptrack-log</code>	Writes a structured session log + handoff file at end of session
<code>/review-paper</code>	Runs a 6-agent pre-submission referee report on your paper
<code>/latex-tables</code>	Formats publication-ready $\text{\LaTeX}$ tables with booktabs

4.7. Agents — Parallel Workers

What is an agent?

An agent is a **separate Claude instance** that runs in its own context window, independently from your main conversation. While a skill guides Claude within *your* conversation, an agent works *for you* in a separate process and reports back when it is done.

You can spawn multiple agents simultaneously. For example, while you continue working in the main session, three agents could be running in the background:

- One reviewing your code for errors

- One cleaning a dataset
- One drafting slides from your results

Each runs independently, in parallel, using its own context window and consuming its own tokens.

Skills vs. agents — the key distinction

	Skills	Agents
Where	In your conversation	Separate process
How invoked	<code>/skill-name</code>	Spawned via the Agent tool
Context	Shares your context window	Has its own context window
Parallelism	Sequential (one at a time)	Multiple can run in parallel
Best for	Consistent, repeatable workflows	Complex, multi-step background tasks

Mental model: Skills are *recipes* (instructions for the chef). Agents are *sous-chefs* (separate workers you delegate to). In setting up the agent you can assign him one or more skills to use but you can also give him custom instructions — for example, to follow specific coding conventions or to save files in a certain way. The agent becomes a mini-version of Claude that works in parallel with its own context and tools.

How to set up a custom agent

Create a `.md` file in `~/.claude/agents/`. The file starts with a header block (between `---` markers) specifying the agent's name, description, model, and available tools, followed by the agent's instructions:

```
---
name: mr-data-setup
description: Data cleaning, merging, reshaping in Stata.
model: sonnet
tools: [Read, Write, Bash, Glob, Grep]
---

You are a data pipeline specialist. Your job is to clean,
merge, reshape, and construct variables in Stata.

## Conventions
- Raw data is sacred -- never modify it
- Save intermediate files to Data/Intermediate/
- Save final files to Data/Final_Data/
- Use clear variable labels
- Log all operations
```

Built-in vs. custom agents

Claude Code comes with several built-in agent types you can use immediately without creating any files:

- **Explore:** Fast codebase exploration — find files, search code, answer questions about project structure.
- **Plan:** Design implementation strategies before writing code.
- **general-purpose:** Flexible agent for research, web search, and multi-step tasks.

Custom agents (like the example above) add your domain-specific knowledge and conventions on top of these capabilities.

Example custom agents

Agent	What it does
mr-data-setup	Executes data cleaning/merging tasks in Stata autonomously
referee	Anonymous peer reviewer calibrated to top journals
beamer-builder	Builds Beamer presentations in your exact style
lit-review	Literature scout — searches web, identifies key papers, synthesizes findings

4.8. Rules — Auto-loaded Protocols

What are rules?

Rules are `.md` files stored in `~/.claude/rules/`. They are loaded **automatically** at the start of every session — you never need to invoke them. Unlike skills (which you trigger manually with `/name`) and agents (which you spawn for specific tasks), rules run silently in the background, shaping how Claude approaches all of your work.

How are rules different from `CLAUDE.md`?

	CLAUDE.md	Rules
Scope	Master config — identity, rules, projects	Modular — one protocol per file
Location	<code>~/.claude/CLAUDE.md</code> or <code>project/.claude/CLAUDE.md</code>	<code>~/.claude/rules/*.md</code>
Loading	Always loaded	Always loaded
Purpose	Tell Claude <i>who you are</i>	Tell Claude <i>how to work</i>

Think of `CLAUDE.md` as your constitution and rules as your standard operating procedures.

Example rules

Rule file	What it does
plan-first-workflow.md	For any non-trivial task, enter plan mode before writing code. Draft plan, save to disk, get approval, then implement.
orchestrator-protocol.md	After a plan is approved, execute autonomously: implement, verify, review, fix, score. Maximum 5 review-fix rounds.
verification-protocol.md	Before declaring a task complete, verify: Stata syntax and paths, $\text{\LaTeX}$ compilation, output file existence.

4.9. The full directory tree

Now that you know all the pieces — `CLAUDE.md`, settings, skills, agents, and rules — here is how they fit together on disk:

```

Macintosh HD/                                (your system disk)
Users/yourname/
  .claude/                                    Claude's home base
  CLAUDE.md                                  Global rules
  settings.json                              Permissions & hooks

OneDrive/                                    (or Dropbox, Google Drive, etc.)
  .claude; Your customizations (synced)
  skills/                                    Reusable playbooks
  agents/                                    Parallel workers
  rules/                                    Auto-loaded protocols

Project_1/                                    Your research projects
  .claude/
  CLAUDE.md                                  Project-specific context
  settings.local.json                        Project permissions
Data/
Codes/
Project_2/
  .claude/
  CLAUDE.md
Project_3/
  .claude/
  CLAUDE.md

```

Notice that skills, agents, and rules live in your cloud storage folder (OneDrive, Dropbox, etc.) rather than directly inside `~/claude/`. This way your customizations are backed up and synced across machines. To make this work, you create **symlinks** — shortcuts that tell Claude Code to look in your cloud folder instead. This is a one-time setup per machine. If you are not familiar with symlinks, just ask Claude in a chat session: *“Help me set up symlinks so my skills, agents, and rules sync via OneDrive.”* It will walk you through the exact commands for your system.

5. Workflows & Best Practices

5.1. Session Continuity — Log Files and Handoffs

Everything you set up in your `CLAUDE.md` (your identity, rules, project structure) persists across sessions. But Claude does **not** remember what you *did* in a previous session. It knows who you are and what project you are working on, but not that yesterday you fixed the merge issue, or that the regression tables are done and you still need robustness checks. That session-specific context is lost every time you start a new conversation.

Log files bridge this gap. They capture what happened in a session so that the next session can pick up where you left off, without you having to re-explain anything.

The solution: a log-writing skill

Create a skill (or use one like `/keeptrack-log`) that writes a structured log file at the end of each session. The skill produces two files:

1. **Session log** (`notes/session_2026-03-25.md`): A detailed record of what was done, what decisions were made, and what follow-ups remain.
2. **Handoff file** (`.claude/handoff.md`): A condensed snapshot optimized for the next session's cold start. Just enough context for Claude to pick up exactly where you left off.

How it works in practice

End of session:

```
You: "Let's wrap up. Write the session log."  
Claude: [writes session log + updates handoff.md]
```

Start of next session: Your CLAUDE.md tells Claude to read the handoff file as part of the session checklist. Claude reads it and says: *“Last session (March 25): we fixed the merge issue in 1a-Cleaning.do and ran the baseline regressions. Open follow-ups: robustness checks with alternative clustering.”*

No re-explaining needed. The handoff file bridges sessions automatically.

Why this approach works

- **Explicit:** you can read and edit the log yourself. It is a plain Markdown file.
- **Versioned:** one file per session, nothing gets overwritten.
- **Human-readable:** useful for your own notes, not just Claude's.
- **Persistent:** survives across machines, context resets, and weeks of inactivity.

5.2. Privacy & Data Protection

Claude Code has file access: it can read and modify files on your machine. This is what makes it powerful, but it also means you need to control what it can touch. There are three layers of protection, from softest to hardest.

Layer 1: CLAUDE.md rules (instructional)

Add clear instructions to your CLAUDE.md:

```
## Critical Rules  
1. NEVER delete data files (.dta, .xlsx, .csv)  
2. NEVER delete program files (.do, .R, .py)  
3. Raw data is sacred -- NEVER modify files in Data/Raw_Data/  
4. Ask before deleting any file
```

Claude follows these rules reliably, but they are instructional, not technically enforced by the system. Think of them as strong suggestions that Claude takes very seriously.

Layer 2: PreToolUse hooks (programmatic)

A hook is a shell script that runs automatically *before* every tool call. If the script exits with code 2, the tool call is blocked. This gives you programmatic control over what Claude can do.

Example hook (`~/claude/hooks/enforce-boundary.sh`):

```
#!/bin/bash  
# Block any read/write to Raw_Data directories  
if echo "$TOOL_INPUT" | grep -q "Raw_Data"; then  
  echo "BLOCKED: Cannot access Raw_Data."  
  exit 2  
fi  
exit 0
```

You configure it in `settings.json` under the `hooks` section.

Layer 3: Permission deny rules (strongest)

The strongest protection: a hard block in `settings.json` that **cannot be overridden**, not even by Claude itself:

```
{
  "permissions": {
    "deny": [
      "Read(/Raw_Data/**)",
      "Write(/Raw_Data/**)",
      "Edit(/Raw_Data/**)"
    ]
  }
}
```

If you want to setup this privacy layers you can ask Claude directly and test it yourself trying to trigger the block. For example, after setting up the deny rules, ask Claude to read a file in `Raw_Data/` and see that it is blocked.

5.3. Best Practices

Eight practices that make the difference

Setup:

1. **Write a good CLAUDE.md.** Have Claude help you write it. Thirty minutes upfront saves hours of repeated instructions.
2. **Set clear boundaries.** Protect raw data with all three layers. Review changes before committing.
3. **Use Skills.** Create reusable playbooks for recurring tasks. Do not re-explain your conventions every session.
4. **Use Git (GitHub).** Git is a version control system that keeps a full history of every change to your files. If Claude Code ever messes something up (overwrites a script, breaks a do-file), you can revert to any previous version instantly. Claude can handle commits for you directly from the terminal. If you are not familiar with Git, ask Claude to walk you through the setup.

Workflow:

5. **Log files for continuity.** Use a session-logging skill to write handoff files. Claude picks up where you left off (see Section 8).
6. **Plan first, act second.** Plan in one session, implement in a fresh one. Long threads degrade performance. Fresh context = better output.
7. **Use Claude to learn Claude.** Ask first, understand, then execute. Claude is the best teacher of its own capabilities.
8. **Trust but verify.** Always check outputs, especially regressions. Claude is powerful but not infallible. Run the code, inspect the results.

Plan mode — think before you act

For any non-trivial task, **enter plan mode before writing code**. Plan mode is a read-only exploration phase where Claude investigates your codebase, designs an approach, and presents it for your approval, without making any changes to your files.

Why it matters:

- Prevents wasted effort on wrong approaches
- Catches ambiguity and misunderstandings before implementation begins

- Plans are saved to disk and survive context compression
- Fresh context after planning produces better output

Example — how a plan mode interaction looks:

```

You: I need to add heterogeneity analysis to my LP
regressions. Let's enter plan mode first --
I want to align on the approach before we
start coding.

Claude: [enters plan mode]
[explores codebase, reads existing regression files]
[drafts plan: which files to modify, which
interactions to add, output format]
[presents plan for your approval]

You: Looks good, but use revenue instead of employees
for firm size.

Claude: [updates plan, exits plan mode, implements]

```

Plan files are saved to `notes/plan_YYYY-MM-DD_description.md`. You can reference them in future sessions and they serve as documentation of your design decisions.

6. A Complete Example: Guido's Setup

Here is how all the pieces fit together in a real research workflow.

6.1. The multi-layer flow in action

Step 1 — Session starts. Claude reads `~/.claude/CLAUDE.md` and learns: Guido is a PhD candidate in economics at IHEID. Primary tools: Stata, $\text{\LaTeX}$, R. Critical rules: never delete data, never delete programs, raw data is sacred. Active projects: JM Paper, Bank Ownership, Bond Level Paper.

Step 2 — Claude asks: *“Which project folder are we working in today?”*

Step 3 — Guido says: “JM Paper”

Step 4 — Claude reads `1-JM Paper/.claude/CLAUDE.md` and learns the project boundary (stay within this folder), the research focus (sovereign-bank-firm transmission in Portugal), and the key context files to read.

Step 5 — Claude reads the handoff file and key context documents. Now it knows the full state: what was done last session, what is pending, what the research design is.

Total time: approximately 30 seconds. No re-explaining needed.

6.2. Directory structure

```

~/.claude/
CLAUDE.md          Global rules (always loaded)
settings.json      Permissions + hooks
skills/ → OneDrive  Symlinked for cloud sync
agents/ → OneDrive  Symlinked for cloud sync
rules/ → OneDrive   Symlinked for cloud sync
~/OneDrive/

```

```

1-JM_Paper/
.claude/
CLAUDE.md      Project-specific context
handoff.md     Last session's state
Data/
Codes/
Drafts/
notes/         Session logs + plans
2-Bank_Ownership/
.claude/CLAUDE.md
3-Bond_Level_Paper/
.claude/CLAUDE.md

```

6.3. Selected skills, agents, and rules

Type	Name	Purpose
<i>Skills</i>		
	<code>/stata-regression</code>	Run <code>reghdfe</code> panel regressions, export <code>.tex</code> tables
	<code>/notes-log</code>	End-of-session log + handoff file
	<code>/latex-tables</code>	Generate publication-ready L ^A T _E X tables
	<code>/latex-beamer</code>	Create Beamer slides from paper content
<i>Agents</i>		
	<code>mr-data-setup</code>	Data cleaning/merging in Stata — executes autonomously
	<code>referee</code>	Anonymous peer reviewer calibrated to top journals
	<code>beamer-builder</code>	Builds Beamer presentations in exact style conventions
	<code>lit-review</code>	Literature scout — web search, synthesis, positioning
<i>Rules</i>		
	<code>plan-first-workflow</code>	Plan before coding; save plans to disk
	<code>orchestrator-protocol</code>	Autonomous execution after plan approval
	<code>pdf-processing</code>	Handle PDF reading and extraction tasks

7. Pricing & Token Consumption

Claude Code charges per token. Every file read, every tool call, every agent spawn, and every response consumes tokens. Understanding this helps you use Claude Code efficiently.

7.1. Choosing a model

Claude Code lets you choose which model runs under the hood. You can switch models at any time, either in your settings or by typing `/model` during a session. The main options:

Model	Speed	Best for
Haiku	Fastest	Quick edits, simple tasks, cheap
Sonnet	Balanced	Daily workhorse — good quality, reasonable cost
Opus	Slowest	Complex reasoning, multi-step tasks, highest quality

The model you choose is the single biggest factor in cost. Opus is roughly 5x more expensive than Sonnet, and Sonnet is roughly 4x more expensive than Haiku. For most tasks, Sonnet is the sweet spot. Use Opus when you need deep reasoning (e.g., debugging a tricky Stata loop, designing an identification strategy) and Haiku for quick, mechanical tasks.

7.2. Two ways to pay

Option 1: API key (pay-per-token). You create an Anthropic API key and pay for exactly what you use. Input tokens (what Claude reads) and output tokens (what Claude writes) are priced separately.

Option 2: Subscription (claude.ai). Claude Code is bundled into Claude’s subscription plans with usage allowances:

Plan	Price/month	Claude Code access
Pro	\$20	Limited or no access
Max 5x	\$100	Included — moderate usage cap
Max 20x	\$200	Included — high usage cap
Team	\$30/user	Limited access
Enterprise	Custom	Custom limits

For heavy users, the Max plans are usually more cost-effective than API pricing.

Prices as of early 2026 — check <https://www.anthropic.com/pricing> for current rates.

7.3. What consumes the most tokens?

1. **Model choice.** This is the biggest lever. Running Opus instead of Sonnet multiplies your cost by ~5x for the same task. Always ask yourself: does this task really need Opus?
2. **Large file reads.** The longer the file, the more tokens. Format matters too: PDFs are especially expensive because they are converted to images internally, so a 10-page PDF costs far more than the same text in a `.txt` or `.md` file.
3. **Conversation history.** Every message you send includes the *entire* conversation history up to that point. A session that has been running for an hour re-sends everything from the beginning with each new turn. This is why long sessions get progressively more expensive. Start fresh sessions or clear history when the context gets long.
4. **Agent spawns.** Each agent gets its own context window with a copy of the system prompts and instructions. Spawning multiple agents in parallel is powerful but token-heavy.
5. **CLAUDE.md overhead.** Your global `CLAUDE.md` is loaded with every single message. If it is long, that cost adds up fast. Keep it concise.
6. **Extended thinking.** When Claude faces a complex problem, it reasons internally before responding. This “thinking” process can generate thousands of tokens that you do not see but still pay for. It is what makes Opus powerful, but also expensive.

7.4. Tips to reduce costs

1. **Use Sonnet as your default.** Reserve Opus for complex reasoning tasks.
2. **Use plan mode.** Align on the approach first; avoid wasted implementation cycles.
3. **Start fresh sessions** when context gets long. Fewer tokens re-sent per message.
4. **Keep CLAUDE.md concise.** It is loaded with every single message.
5. **Be specific in requests.** Reduce back-and-forth; each turn re-sends the full history.
6. **Use skills.** Conventions are loaded once when triggered, not re-explained every time.

8. Resources & Community

8.1. Official documentation

- Claude Code documentation: <https://docs.anthropic.com/en/docs/claude-code>
- Prompt engineering guide: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering>
- Anthropic pricing: <https://www.anthropic.com/pricing>

8.2. Learning & training

- Anthropic courses: <https://anthropic.skilljar.com/>
- Panjwani's Claude Code for Economists cohort: <https://ai-mba.io/cohorts/claude-code-economists>
- AI for Econ newsletter: <https://ai-mba.io/newsletter/econ>
- Blattman's tools and guides: <https://claudeblattman.com>
- Brunnermeier's "Claude Code for Applied Economists": <https://markusacademy.substack.com/p/claude-code-for-applied-economists>

8.3. Skills & workflows

- Sant'Anna's Claude Code workflow: <https://github.com/pedrohcg/claude-code-my-workflow>
- Anthropic's official skill creator: <https://github.com/anthropics/skills>
- Stata skill setup tutorial: <https://ai-mba.io/tutorials/how-to-create-a-stata-skill>

8.4. People to follow on X

Name	Handle
Alex Imas	@alexolegimas
Chris Blattman	@cblatts
David Yanagizawa-Drott	@YanagizawaD
Ethan Mollick	@emollick
Soumitra Shukla	@soumitrashukla9
Kevin Bryan	@Afinetheorem
Andy Hall	@ahall_research
Pedro Sant'Anna	@pedrohcg
Scott Cunningham	@causalinf
Aniket Panjwani	@aniketpanjwani